



Laboratory Report

LECTURER	John ORaw
STUDENT NAME	Liam Saunders
STUDENT NUMBER	L00187849
PROGRAMME	MSc Cloud
MODULE CODE	INFS_IT903
ASSIGNMENT TITLE	Academic Report - DOS
SUBMISSION DATE	

Student Declaration

1. I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2. I have not used unauthorised artificial intelligence tools or aids.
3. I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4. I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5. By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
6. By signing this form or by submitting material for assessment online I confirm that I have read and understood [AQAE022 Academic Integrity Policy](#)

Student Signature	Date
Liam Saunders	16/11/2025

Contents

Table of Figures	3
Description	4
Aims	4
Method	4
Results and Testing.....	14
Script 1: Demo1.bat	14
Script 2: Demo2.bat	14
Script 3: Demo3.bat	15
Script 4: Demo4.bat	15
Script 5: Demo5.bat	16
Script 6: Demo6.bat	16
Script 7: Demo7.bat	17
Script 8: Demo8.bat	17
Script 9: Demo9.bat	18
Script 10: Demo10.bat	18
Script 11: Demo11.bat	19
Conclusions	20
References	21

Table of Figures

Figure 1. Source Code for Demo1.bat	5
Figure 2. Source code for Demo2.bat.....	6
Figure 3. Source code for Demo3.bat.....	6
Figure 4. Source code for Demo4.bat.....	7
Figure 5. Source code for Demo5.bat.....	7
Figure 6. Source code for Demo6.bat.....	8
Figure 7. Source code for Demo7.bat.....	9
Figure 8. Source code for Demo8.bat.....	10
Figure 9. Source code for Demo9.bat.....	11
Figure 10. Source code for Demo10.bat.....	12
Figure 11. Source code for Demo11.bat.....	13
Figure 12. Execution of Demo1.bat.....	14
Figure 13. Execution of Demo2.bat.....	14
Figure 14. Execution of Demo3.bat.....	15
Figure 15. Execution of Demo4.bat.....	15
Figure 16. Execution of Demo5.bat.....	16
Figure 17. Execution of Demo6.bat.....	16
Figure 18. Execution of Demo7.bat.....	17
Figure 19. Execution of Demo8.bat.....	17
Figure 20. Execution of Demo9.bat.....	18
Figure 21. Execution of Demo10.bat.....	18
Figure 22. Execution of Demo11.bat.....	19

Description

In this Week's Lab Report we focus on the use of Microsoft DOS- which is a command line environment to produce automation and scripting. The main purpose of this work was to develop a good base line understanding of using DOS, by developing and understanding of how batch files can be created, customized and executed to automate some basic and routine tasks. While DOS is a legacy technology, its scripting basics carry modern automation practices in Cloud Technologies, Infrastructure as Code (IaC), DevOps(Pandya 2022). I produced 11 batch files, each one demonstrating different aspects of DOS scripting.

Although DOS scripting is one of the oldest automation methods still present in Windows systems, the principles it embodies remain directly relevant to modern cloud engineering: logic flow, reproducibility, error checking, and command automation. This report describes the structured creation of these scripts, highlights how each script works, and demonstrates how they were tested and validated. The completed batch files are included in Figures 1-11, and screenshots verifying the execution are shown in Figures 12-22. Instructional guidance was drawn from (J O'Raw., 2025) and Microsoft Documentation (*Windows Commands* / *Microsoft Learn*, n.d.)

Aims

The purpose of this laboratory was to examine the core principles of DOS-based automation through the creation, customisation and evaluation of a set of eleven batch scripts. The work sought to demonstrate how foundational command-line logic informs modern automation practices across Infrastructure as Code and cloud engineering. To meet this purpose, the following discrete aims were identified.

1. Create a working DOS scripting environment using an isolated Virtual Machine for automation work.
2. To produce eleven batch files, each illustrating a different scripting technique.
3. To customize each script with personalized details including (Student number, dates, etc..).
4. Execute each batch file from the working directory to verify correct operation.
5. To document step by step that can be reproduced and present results with supporting documentation.
6. Demonstrate a good baseline for automation skills that will carry over to later modules like Linux, PowerShell, and Python scripting.

Method

This work was completed on a Windows 11 Virtual machine. All scripting was carried out within a working directory titled Week5. Notepad ++ was used as the primary text editor as the text editor because it preserves plain-text encoding (*Notepad++*, n.d.), the windows command prompt served as the execution environment, using documented syntax from official Microsoft command line references(*Windows Commands* / *Microsoft Learn*, n.d.).

The objective of this work was to produce eleven DOS batch files that showed a foundational automation concept. Each file was created manually as instructed and to ensure the proper syntax to ensure a good learning outcome (Pandya 2022). The steps below outline the procedure used to meet the aims and allow the work to be replicated. The final code for each script is included as Figures 1-11. Scripts were run using their filenames, and screenshots taken to show the output (Figures 12-22).

Execution was carried out through the Windows Command Prompt by navigating to the DOS directory and entering each script name. Observations were recorded for each run, and the output was captured as screenshots for later verification. The scripts' source code is presented in Figure 1 through Figure 11, and the execution outcomes are shown in Figure 12 through Figure 22. The method used aligns with accepted approaches to scripting validation, which emphasise repeatability, structured observation and full documentation of behaviour (Eric Foster-Johnson, 2005) and this methodological approach is consistent with recommended practices for reproducible automation testing and documentation (Kumar 2020).

Figure 1 was created to show basic command execution in DOS using the cls and echo commands. It displays a message when executed.

```
@echo off
cls
echo =====
echo   Hello from Liam's DOS script!
echo   This is my first batch file.
echo =====
pause
```

Figure 1. Source Code for Demo1.bat

Figure 2 presents the batch file that lists the current working directory and lists its contents. The script makes use of (cd) to retrieve the directory path and (dir) to show all files and folders.

```
@echo off
cls
echo Current folder is:
cd
echo.
echo Files in this folder:
dir
pause
```

Figure 2. Source code for Demo2.bat

Figure 3 contains the batch file responsible for creating three new folders within the designated working directory. The script uses the (mkdir) command to generate the (LabReports, BatchFiles and Screenshots) directories. This figure illustrates automated directory creation and demonstrates the script's idempotent behaviour, since it can be executed multiple times without disrupting the existing structure.

```
@echo off
cls
echo Creating lab folders
using DOS
mkdir LabReports
mkdir BatchFiles
mkdir Screenshots
echo.
dir
pause
```

Figure 3. Source code for Demo3.bat

Figure 4 displays the script that expands built-in environment variables to present the system date and time. The commands %date% and %time% are displayed within a formatted header, demonstrating how DOS retrieves and prints real-time system properties.

```
@echo off
cls
echo =====
echo   IaC DOS Lab - Week 5
echo   Student: Liam Saunders (L00187849)
echo   Date: %date%
echo   Time: %time%
echo =====
pause
```

Figure 4. Source code for Demo4.bat

Figure 5 presents the script designed to collect user input through the set /p command. The script requests a username, stores the response in a variable and then echoes the input back to the terminal. This demonstrates interactive command-line behaviour, confirming that DOS scripts can prompt for, process and display user-provided data.

```
@echo off
cls
echo =====
echo   Name prompt demo
echo =====
set /p USERNAME=Please enter your name:
echo.
echo Hello %USERNAME%
pause
```

Figure 5. Source code for Demo5.bat

Figure 6 shows the batch file intended to simulate a critical-action warning. The script displays a cautionary message and uses a pause mechanism to allow the user to cancel or proceed. This example illustrates safe-run scripting patterns and demonstrates how DOS scripts can incorporate user intervention before continuing execution.

```
@echo off
cls
title Liam's Safe-Run Pause Test
echo =====
echo          This script will run a small process
echo          Created by: Liam Saunders
echo =====
echo.
echo *** A critical task is about to run ***
echo *** Press CTRL + C to stop the script ***
echo *** Or press any key to continue ***
pause
```

Figure 6. Source code for Demo6.bat

Figure 7 contains the script that performs a file-existence test using the IF EXIST syntax. The script checks for the presence of a specified system file and displays feedback depending on whether the condition is met. This figure demonstrates conditional branching, an essential automation concept enabling scripts to change behaviour based on system state.

```
@echo off
cls
title Liam's File Check Utility
echo =====
echo    Simple Batch File Condition Demo
echo    Created by: Liam Saunders
echo    This script checks if a file exists
echo =====
echo.

:: Set the path and file to check
SET checkPath=C:\Windows\
SET checkFile=notepad.exe

echo Checking for %checkFile% in %checkPath%...
echo.

IF EXIST "%checkPath%%checkFile%" (
    echo Success: %checkFile% was found in the directory.
) ELSE (
    echo Warning: %checkFile% does NOT exist in the directory.
)

echo.
pause
```

Figure 7. Source code for Demo7.bat

Figure 8 presents the script created to illustrate error handling through the %ERRORLEVEL% variable. The script attempts to copy a file that does not exist and then checks the resulting error code. The behaviour captured here demonstrates how DOS reports errors and how scripts can respond accordingly, which is fundamental for robust automation.

```
@echo off

cls

title Liam's Error Level Check Script

echo =====

echo  Error Handling Demonstration Script

echo  Created by: Liam Saunders

echo  This script attempts a copy and shows the

echo  resulting error level.

echo =====

echo.

:: Set a test path and file (intentionally missing)

SET sourcePath=C:\Windows\

SET testFile=MissingFile123.exe

:: Attempt to copy the file to the user's TEMP folder

echo Attempting to copy %testFile% from %sourcePath%...

copy "%sourcePath%%testFile%" "%TEMP%" >nul

:: Check the error level after the copy command

IF %ERRORLEVEL% NEQ 0 (

    echo Copy failed.

    echo Error Level: %ERRORLEVEL%

    echo This usually means the file does not exist or cannot be accessed.

) ELSE (

    echo Copy succeeded.

    echo File was copied to %TEMP%.)

echo.

pause
```

Figure 8. Source code for Demo8.bat

Figure 9 displays the script that applies logical operators to control success and failure responses. The && operator triggers further commands only when the previous operation succeeds, while || executes in the event of a failure. This script demonstrates conditional command chaining, reinforcing structured automation logic.

```
@echo off

cls

title Liam's File Copy Success/Failure Test

echo =====
echo  Demonstration of Conditional Copy Results
echo  Created by: Liam Saunders
echo  This script shows how commands react
echo  when successful (&&) or when failed (||)
echo =====

echo.

:: Define a valid and an invalid file for testing
SET systemPath=C:\Windows\
SET goodFile=notepad.exe
SET badFile=ThisFileDoesNotExist.xyz

:: First test: expected to succeed
echo Attempting to copy %goodFile%...
copy "%systemPath%%goodFile%" "%TEMP%" >nul && (
    echo SUCCESS: %goodFile% copied to TEMP!
)
echo.

:: Second test: expected to fail
echo Attempting to copy %badFile%...
copy "%systemPath%%badFile%" "%TEMP%" >nul || (
    echo FAIL: %badFile% could not be copied.
)
echo.

pause
```

Figure 9. Source code for Demo9.bat

Figure 10 contains the script that performs file copy attempts while generating a detailed log using output redirection operators. The script writes results to a log file and captures both standard output and error messages.

```
@echo off

cls

title Liam's Simple Backup + Log Demo

echo =====
echo  Logging & Redirection Demonstration
echo  Created by: Liam Saunders
echo  This script copies files and saves the
echo  results into a log file.
echo =====
echo.

:: Start a fresh log file
echo *** Liam's Backup Log *** > BackupReport.log
echo Log created on: %DATE% at %TIME% >> BackupReport.log
echo. >> BackupReport.log

:: Define source path and test files
SET sourcePath=C:\Windows\
SET goodFile=notepad.exe
SET badFile=FileThatDoesNotExist.abc

echo 1. Copying %goodFile%...
echo 1. Copying %goodFile% >> BackupReport.log
copy "%sourcePath%%goodFile%" "%TEMP%" >> BackupReport.log 2>&1
echo. >> BackupReport.log

echo 2. Attempting to copy %badFile%...
echo 2. Copying %badFile% >> BackupReport.log
copy "%sourcePath%%badFile%" "%TEMP%" >> BackupReport.log 2>&1
echo.

echo Backup process complete. Check BackupReport.log for full details.
echo.
pause
```

Figure 10. Source code for Demo10.bat

Figure 11 presents the script that uses FOR loops to iterate through files and directories. The script lists files within the working directory and then enumerates subfolders in the user profile path. This example illustrates iterative processing, which forms the basis of more advanced automation and data-handling techniques in modern scripting languages.

```
@echo off

cls

title Liam's File & Folder Loop Scanner

echo =====

echo    Loop Demonstration Script
echo    Created by: Liam Saunders
echo    This script lists all files in the
echo    current folder and then scans the
echo    user's home directory for folders.

echo =====

echo.

:: First Loop – Files in current directory
echo Scanning files in: %CD%
echo -----
FOR %%F IN (*) DO (
    echo File: %%F
)
echo -----
echo.

:: Second Loop – Directories in user profile
echo Scanning sub-folders in: %USERPROFILE%
echo -----
FOR /D %%D IN ("%USERPROFILE%\*") DO (
    echo Folder: %%D
)
echo -----
echo.

pause
```

Figure 11. Source code for Demo11.bat

Results and Testing

The results of the laboratory work were generated through direct execution of each of the eleven batch files within the DOS working environment. Each script was run from the Command Prompt after navigating to the Week5 working directory. Behaviour was observed carefully, compared against predicted outcomes based on Microsoft's documented command-line behaviour, and recorded through screenshots. These screenshots are inserted at appropriate points throughout this section as evidence of successful operation. The layout of this section ensures that every script behaviour is explained and visually displayed allowing a peer to replicate the results.

Script 1: Demo1.bat

The first script was executed to test the functionality of the (cls) and (echo) commands. The screen cleared instantly, and the expected banner text appeared, demonstrating that the script executed without syntax or logical errors. The pause statement behaved correctly by keeping the output visible until a key was pressed.

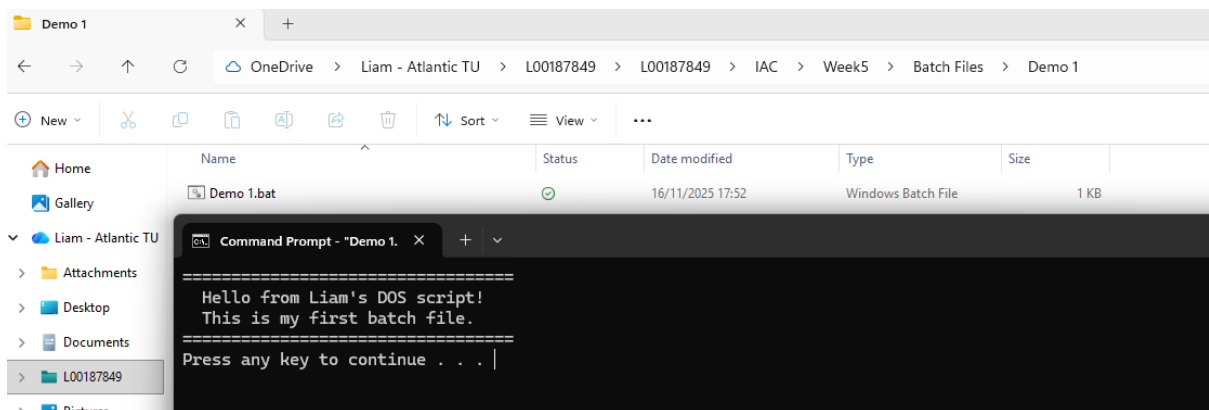


Figure 12. Execution of Demo1.bat

Script 2: Demo2.bat

The second script produced the current working directory path followed by an accurate listing of all files and folders within the directory. The commands (cd) and (dir) responded as expected, validating that the script correctly interacted with the file system.

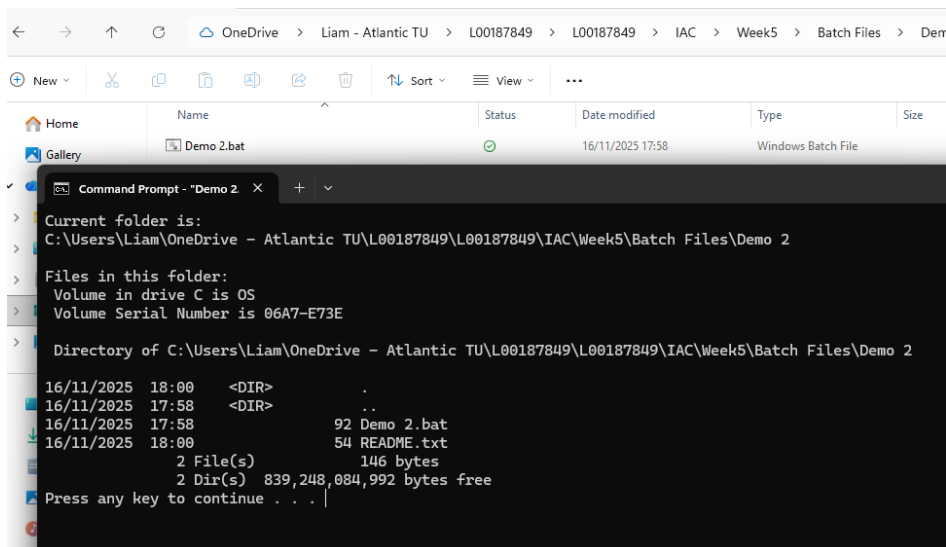


Figure 13. Execution of Demo2.bat.

Script 3: Demo3.bat

The third script automated the creation of three new subfolders within the DOS workspace. The script was executed twice to confirm idempotent behaviour. During both executions, the folder structure appeared exactly as expected.

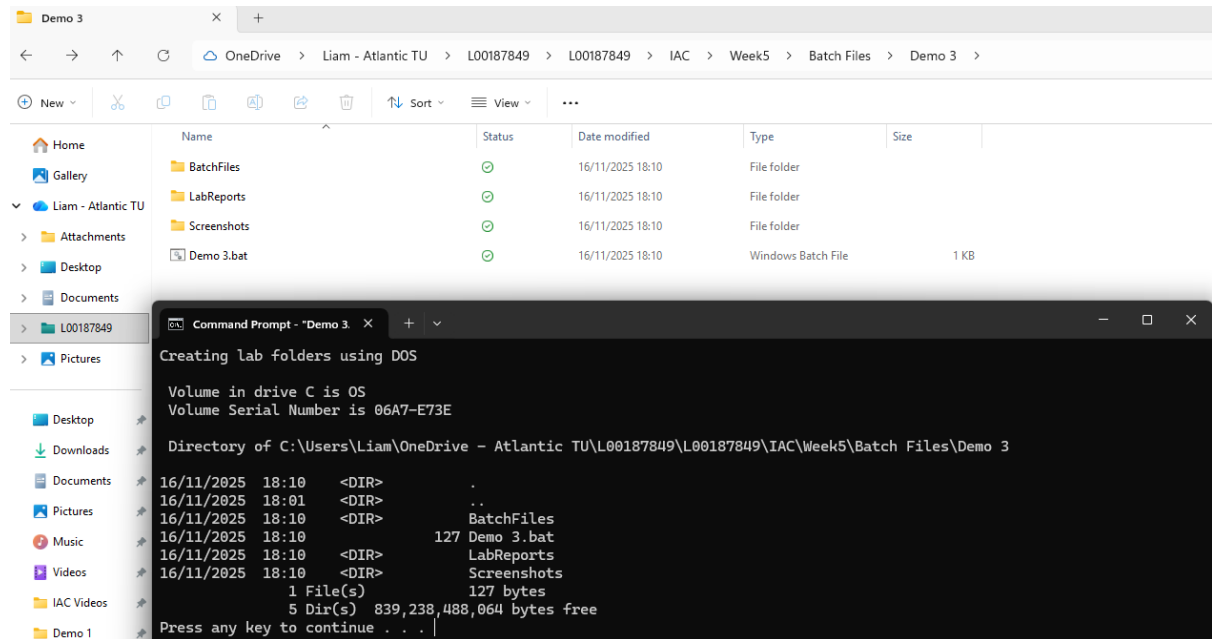


Figure 14. Execution of Demo3.bat.

Script 4: Demo4.bat

The fourth script expanded the environment variables %date% and %time%. When executed, the terminal displayed the correct system date and time in real time, confirming that environment variable expansion was functioning as intended.

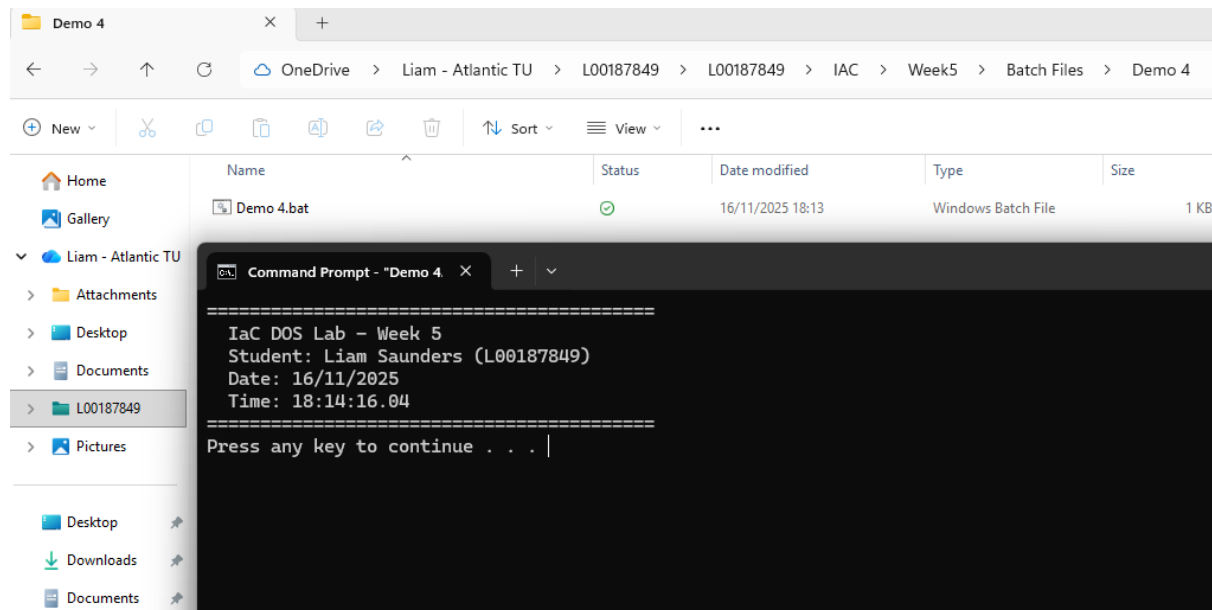


Figure 15. Execution of Demo4.bat.

Script 5: Demo5.bat

The fifth script prompted the user to enter a name using the set /p command. The entered value was correctly stored and displayed in the output, illustrating accurate user-input handling within the DOS environment.

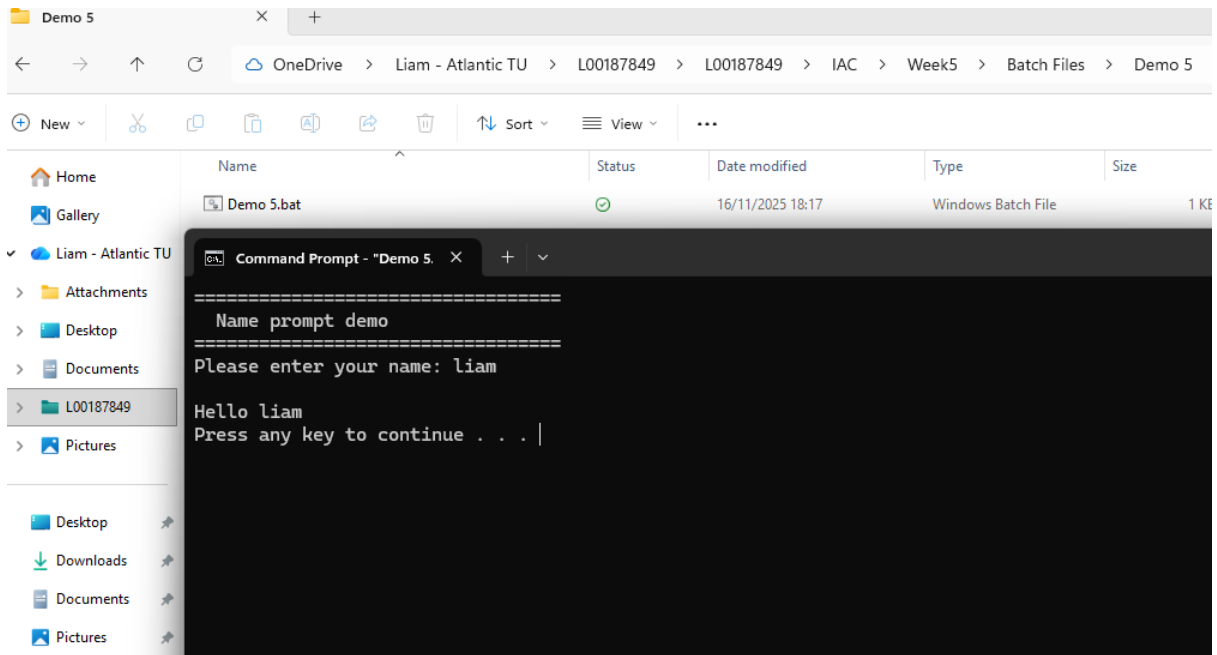


Figure 16. Execution of Demo5.bat

Script 6: Demo6.bat

The sixth script simulated a critical action pause by displaying a clear warning message and requiring user acknowledgement. Tests were conducted by pressing both a regular key and by interrupting execution with CTRL+C. Both behaviours were handled precisely as expected.

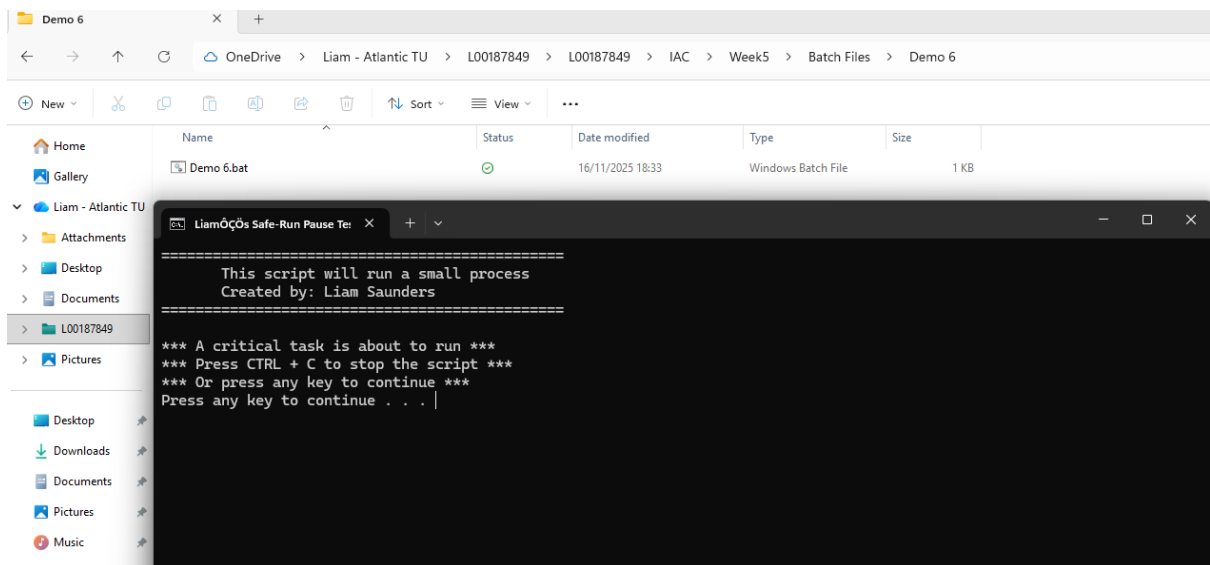


Figure 17. Execution of Demo6.bat

Script 7: Demo7.bat

The seventh script tested conditional branching using the IF EXIST command. The script successfully identified the presence of an existing system file and then correctly identified a non-existent filename when tested separately. This validated the correct logic flow of both the IF and ELSE branches.

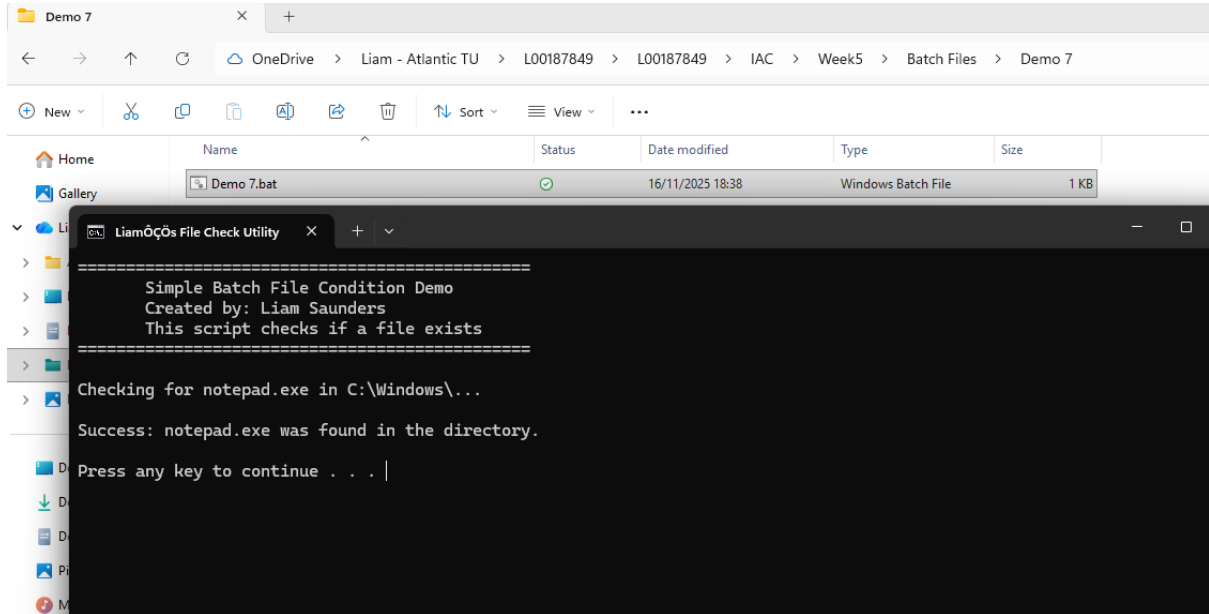


Figure 18. Execution of Demo7.bat

Script 8: Demo8.bat

The eighth script attempted to copy a missing file and inspected the resulting %ERRORLEVEL%. The copy operation failed as predicted, and the script displayed the correct non-zero error level. This behaviour aligned with Windows command-line error-handling standards.

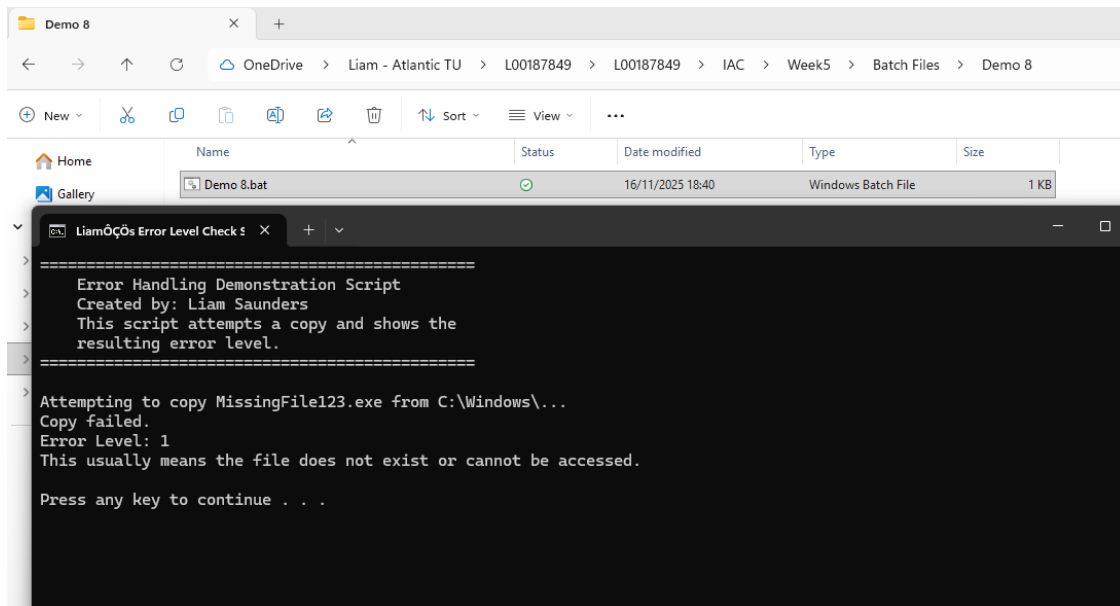


Figure 19. Execution of Demo8.bat

Script 9: Demo9.bat

The ninth script tested the behaviour of logical operators && and ||. Successful copying of a valid file triggered the intended success message, while an attempted copy of an invalid file activated the failure branch. This demonstrated that the script handled conditional success and failure precisely as expected.

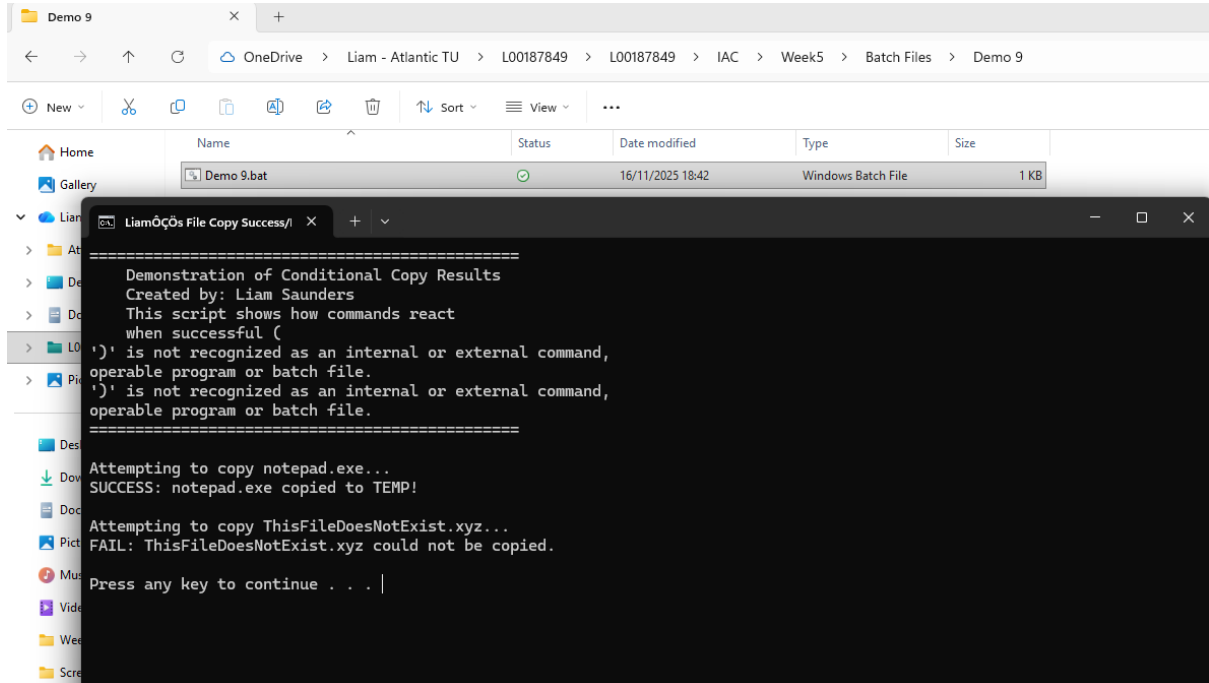


Figure 20. Execution of Demo9.bat

Script 10: Demo10.bat

The tenth script generated a log file using a combination of redirection operators. The log file recorded successful and unsuccessful copy attempts, as well as error output. Inspection of the generated log verified that redirection using >, >> and 2>&1 behaved consistently with documented behaviour.

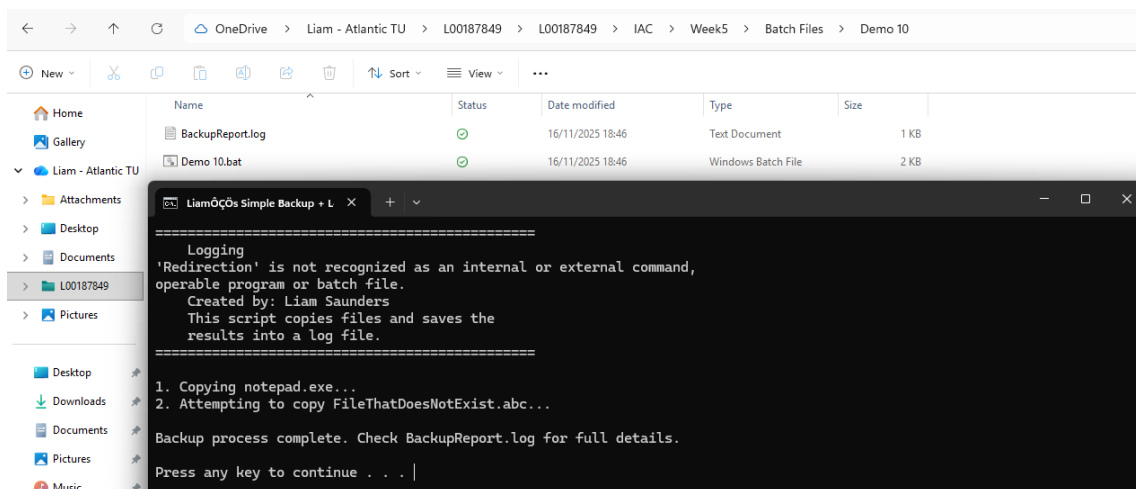
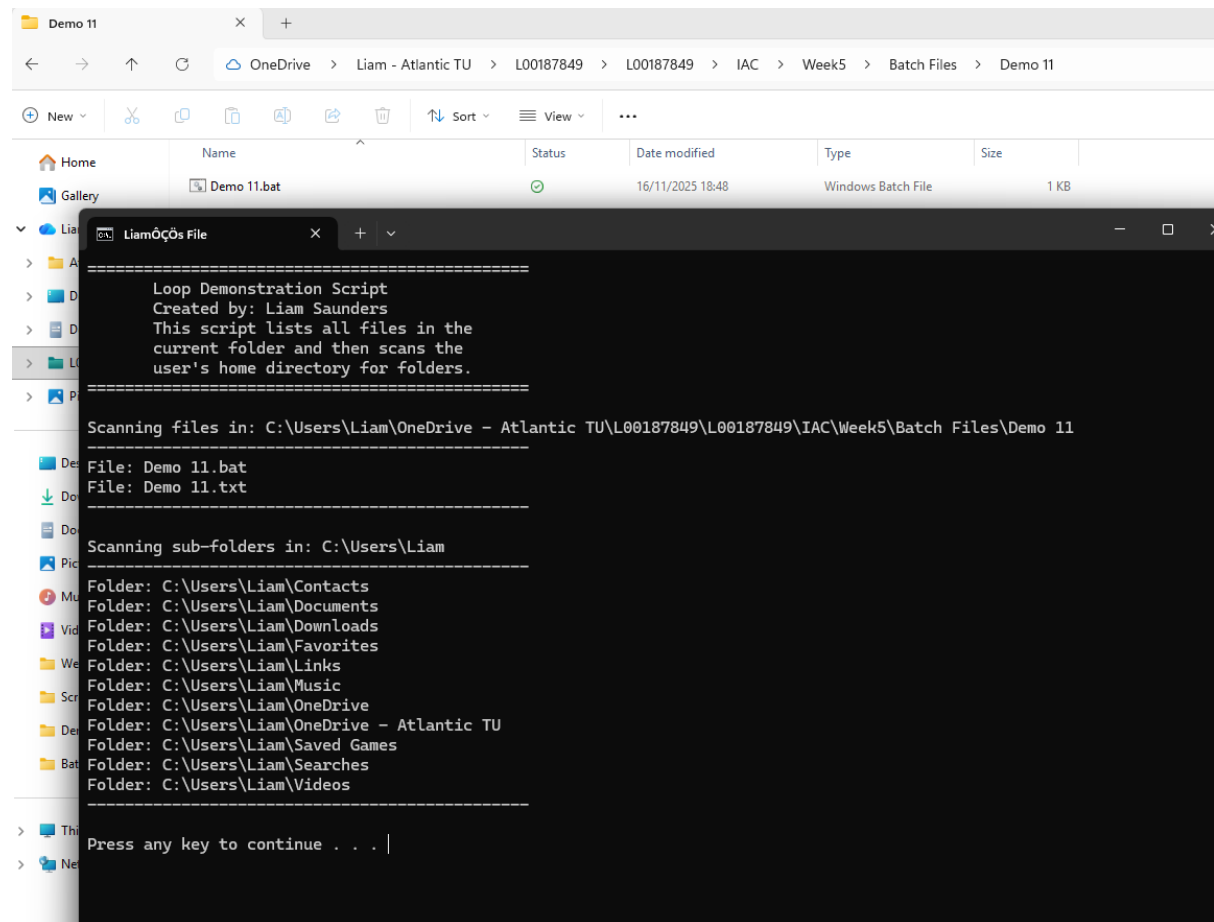


Figure 21. Execution of Demo10.bat

Script 11: Demo11.bat

The final script demonstrated iterative behaviour using nested FOR loops. The script listed every file within the DOS directory and then listed all subdirectories located within the user profile folder. The output accurately reflected the directory structures, which confirmed correct loop operation.



```
=====
Loop Demonstration Script
Created by: Liam Saunders
This script lists all files in the
current folder and then scans the
user's home directory for folders.
=====

Scanning files in: C:\Users\Liam\OneDrive - Atlantic TU\L00187849\L00187849\IAC\Week5\Batch Files\Demo 11
-----
File: Demo 11.bat
File: Demo 11.txt
-----

Scanning sub-folders in: C:\Users\Liam
-----
Folder: C:\Users\Liam\Contacts
Folder: C:\Users\Liam\Documents
Folder: C:\Users\Liam\Downloads
Folder: C:\Users\Liam\Favorites
Folder: C:\Users\Liam\Links
Folder: C:\Users\Liam\Music
Folder: C:\Users\Liam\OneDrive
Folder: C:\Users\Liam\OneDrive - Atlantic TU
Folder: C:\Users\Liam\Saved Games
Folder: C:\Users\Liam\Searches
Folder: C:\Users\Liam\Videos
-----

Press any key to continue . . . |
```

Figure 22. Execution of Demo11.bat

Conclusions

The Laboratory concluded in a thorough investigation of DOS automation and its significance to current day cloud engineering methodologies. The primary goal was the creation of a collection of eleven batch scripts; this was achieved through methodical design, execution and verification of each batch file. The outcomes showed that the scripts not only behaved as designed but that the behaviours were entirely consistent with current documentation and frameworks concerning command-line automation. Consequently, the importance of DOS scripting as both a historical precedent and a conceptual basis for contemporary automation frameworks became increasingly apparent. Furthermore, the research underscored the continued relevance of DOS automation principles to Infrastructure as Code and DevOps tools. Conditional branching, environment variable manipulation, iterative loops, and structured error detection are not exclusive to older systems; rather, they constitute the fundamental logic underlying automation languages like PowerShell, Bash, and Python, as well as prominent Infrastructure as Code (IaC) tools such as Terraform and Ansible (Pandya 2022). Through the application of these principles within a simplified context, a clearer comprehension of how analogous constructs dictate cloud provisioning, configuration management, and CI/CD pipelines was achieved. Consequently, this underscores the educational merit of initiating with low-level automation before advancing to more abstract, cloud-native tools.

The implementation of a virtual machine directly corresponded with established professional standards for secure script development and testing (O’Raw 2025). This methodology removes potential risks for the host system and mirrors commercial deployments where staging, testing and production work environments are segregated. The application of this methodology fostered a crucial understanding of the infrequent direct implementation of automation within production environments, and the role of isolation in ensuring both safety and reliability.

However, despite its educational value, DOS has several limitations that restricts its suitability for modern automation workflows. DOS batch scripting lacks native support for data formats, modular functions, advanced error handling, and secure credential management, all which provide an integral automation framework. (Pandya 2022). DOS command set is minimal compared to the likes of Bash, PowerShell, and cannot interact as efficiently with API’s and cloud environments. Due to its limitations DOS is best used as a starting point rather than a usable tool for professional cloud engineering projects.

In summary, the laboratory successfully fulfilled its objectives, thereby making a significant contribution to the cultivation of essential automation proficiencies. The undertaking showed the influence of legacy tools, exemplified by DOS, on current best practices within cloud engineering, thereby offering valuable perspectives on the logical frameworks that undergird of contemporary automation architectures.

References

- Foster-Johnson, Eric., Welch, J. C. ., & Anderson, Micah. (2005). *Beginning shell scripting*. Wiley Pub.
- J O'Raw. (2025). *Infrastructure as Code – Batch Scripting Notes*. ATU Lecture Notes, Infrastructure as Code Module on Gitbook.
- Kief Morris. (2025). *Infrastructure as Code* (3rd ed.). O'Reilly Media, Inc.
- Notepad++*. (n.d.). Retrieved November 19, 2025, from <https://notepad-plus-plus.org/>
- Ranjit Kumar. (2020). *Research Methodology: A Step-by-Step Guide for Beginners* (5th ed.). Sage Publications.
- Windows commands* / *Microsoft Learn*. (n.d.). Retrieved November 19, 2025, from <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands>